PRACTICAL 7:

```
#include<stdio.h>
int main()
{
   int i,j,k;
   int processes = 5;
   int resources = 4;
   int allocation[5][4] = {
      {0,0,1,4},
      {0,6,3,2},
      {0,0,1,2},
      {1,0,0,0},
      {1,3,5,4}
   };
   int max[5][4] = {
      {0,6,5,6},
      {0,6,5,2},
      {0,0,1,2},
      {1,7,5,0},
      {2,3,5,6}
   };
   int available[4] = {1,6,2,0};
   int need[5][4];
   int finish[5]={0};
   int safeSeq[5];
   int count=0;

   // Calculate and display NEED matrix
   printf("NEED MATRIX\n");
   for(i=0;i<processes;i++)
   {
      for(j=0;j<resources;j++)
      {
         need[i][j]=max[i][j]-allocation[i][j];
         printf("%d ",need[i][j]);
      }
      printf("\n");
   }

   // Display initial available resources
   printf("\nINITIAL AVAILABLE RESOURCES: ");
   for(j=0;j<resources;j++)
   {
      printf("%d ",available[j]);
   }
   printf("\n");

   // Banker's algorithm to find safe sequence
   while(count<processes)
   {
```

```
int found = 0;  // flag to check if any process was allocated in this iteration
for(i=0;i<processes;i++)
{
   if(finish[i]==0)
   {
      int flag=0;
      for(j=0;j<resources;j++)
      {
         if(need[i][j]>available[j])
         {
            flag=1;
            break;
         }
      }
      if(flag==0)
      {
         found = 1;
         safeSeq[count]=i;
         finish[i]=1;

         // Display process being executed
         printf("\nPROCESS P%d EXECUTING:\n", i);
         printf("  Need: ");
         for(j=0;j<resources;j++)
            printf("%d ", need[i][j]);
         printf("\n  Allocation: ");
         for(j=0;j<resources;j++)
            printf("%d ", allocation[i][j]);
         printf("\n  Available before: ");
         for(j=0;j<resources;j++)
            printf("%d ", available[j]);

         // Update available resources
         for(k=0;k<resources;k++)
         {
            available[k]+=allocation[i][k];
         }

         printf("\n  Available after: ");
         for(j=0;j<resources;j++)
            printf("%d ", available[j]);
         printf("\n");

         count++;
      }
   }
}

// If no process could be allocated, break to avoid infinite loop
if(found == 0)
{
```

```
            printf("\nNo safe sequence exists!\n");
            break;
        }
    }

    // Display final safe sequence
    if(count == processes)
    {
        printf("\nSAFE SEQUENCE FOUND:\n");
        for(i=0;i<processes;i++)
        {
            printf("P%d ",safeSeq[i]);
            if(i < processes-1)
                printf("-> ");
        }
        printf("\n");

        // Display final available resources
        printf("\nFINAL AVAILABLE RESOURCES: ");
        for(j=0;j<resources;j++)
        {
            printf("%d ",available[j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT :

```
NEED MATRIX
0 6 4 2
0 0 2 0
0 0 0 0
0 7 5 0
1 0 0 2

INITIAL AVAILABLE RESOURCES: 1 6 2 0

PROCESS P1 EXECUTING:
  Need: 0 0 2 0
  Allocation: 0 6 3 2
  Available before: 1 6 2 0
  Available after: 1 12 5 2

PROCESS P2 EXECUTING:
  Need: 0 0 0 0
  Allocation: 0 0 1 2
  Available before: 1 12 5 2
  Available after: 1 12 6 4

PROCESS P3 EXECUTING:
  Need: 0 7 5 0
  Allocation: 1 0 0 0
  Available before: 1 12 6 4
  Available after: 2 12 6 4
```

```
PROCESS P4 EXECUTING:
  Need: 1 0 0 2
  Allocation: 1 3 5 4
  Available before: 2 12 6 4
  Available after: 3 15 11 8

PROCESS P0 EXECUTING:
  Need: 0 6 4 2
  Allocation: 0 0 1 4
  Available before: 3 15 11 8
  Available after: 3 15 12 12

SAFE SEQUENCE FOUND:
P1 -> P2 -> P3 -> P4 -> P0

FINAL AVAILABLE RESOURCES: 3 15 12 12
```